

کامپایلر NNGraph DSL

یک زبان دامنه‌محور برای توصیف معماری شبکه‌های عصبی
و تولید خودکار کد PyTorch

گزارش فنی پروژه

۱۱ تیر ۱۴۰۵

فهرست مطالب

۳	چکیده
۴	۱ مقدمه
۴	۱.۱ انگیزه
۵	۲.۱ هدف
۵	۳.۱ دامنه کاربرد
۶	۲ کارهای مرتبط
۶	۱.۲ چارچوب NADER
۶	۲.۲ FANTLR
۶	۱.۲.۲ الگوی Listener در برابر Visitor
۷	۳.۲ ابزارها و تجسم‌سازهای AST
۷	۱.۳.۲ چرا AST در اینجا استفاده نمی‌شود
۹	۳ طراحی زبان
۹	۱.۳ ساختار برنامه
۱۰	۲.۳ گرامر
۱۱	۳.۳ سیستم انواع
۱۲	۴.۳ انواع لایه
۱۲	۱.۴.۳ لایه‌های پارامتردار
۱۲	۲.۴.۳ فعال‌سازی‌ها
۱۲	۳.۴.۳ عملیات گراف (بدون ماژول)
۱۳	۴.۴.۳ پارامترهای سراسری
۱۳	۵.۳ نحو یال
۱۳	۶.۳ مثال کامل: MLP
۱۶	۴ پیاده‌سازی
۱۶	۱.۴ معماری کامپایلر

۱۷	مسئولیت فایل‌ها	۱.۱.۴
۱۷	تحلیل‌گر معنایی	۲.۴
۱۷	انتشار مقدار در درخت	۱.۲.۴
۱۸	قوانین اعتبارسنجی	۲.۲.۴
۱۸	قالب پیام خطا	۳.۲.۴
۱۹	FalseCtx_ برای خطاهای پس از پیمایش	۴.۲.۴
۱۹	جریان اجرای شنونده	۵.۲.۴
۲۴	مولد کد	۳.۴
۲۴	الگوریتم تخصیص متغیر	۴.۴
۲۵	ردیابی MLP (زنجیره خطی)	۱.۴.۴
۲۵	ردیابی ResBlock (فناوت سپس ادغام)	۲.۴.۴
۲۶	ردیابی InceptionBlock (دو شاخه موازی)	۳.۴.۴
۲۶	ردیابی TransformerEncoder (دو residual ضمنی)	۴.۴.۴
۲۷	موارد خاص تولید کد	۵.۴.۴
۲۸	ساختار کد خروجی	۶.۴.۴
۲۸	استنتاج شکل	۵.۴
۲۹	تجسم Graphviz	۶.۴
۲۹	داربست آموزش	۷.۴
۲۹	حاشیه‌نویسی کوانتیزاسیون	۸.۴
۲۹	صادرکردن ONNX	۹.۴

۵ ارزیابی

۳۰	نتایج pass forward	۱.۵
۳۱	اثبات درستی متغیرها	۲.۵
۳۱	آمار کدبیس	۳.۵

۶ تصمیمات طراحی

۳۲	Listener در برابر Visitor	۱.۶
۳۲	load_from_listener() در برابر generate(traversal)	۲.۶
۳۲	نام‌گذاری متغیر با node_id	۳.۶

۷ نتیجه‌گیری و کارهای آینده

۳۳	نتیجه‌گیری	۱.۷
۳۳	قابلیت‌های پیاده‌سازی‌شده	۲.۷
۳۴	کارهای آینده	۳.۷

۳۵

منابع

چکیده

NNGraph DSL یک زبان دامنه‌محور است که برای توصیف معماری شبکه‌های عصبی به صورت گراف جهت‌دار طراحی شده است. فایل‌های `nng` را به عنوان ورودی دریافت کرده و کد `PyTorch` اجرایی تولید می‌کند که یک زیرکلاس `nn.Module` کامل با متدهای `__init__` و `forward` است.

کامپایلر با `ANTLR4` نسخه `۲.۱۳.۴` ساخته شده و از الگوی `Listener` برای تحلیل معنایی و الگوریتم مرتب‌سازی توپولوژیک `Kahn` هم در تحلیل‌گر معنایی (تشخیص دور) و هم در مولد کد (ترتیب اجرا) استفاده می‌کند. استنتاج شکل تنسور، تجسم گراف با `Graphviz`، داربست آموزش، حاشیه‌نویسی کوانتیزاسیون و صادرکردن `ONNX` نیز پشتیبانی می‌شوند.

چهار معماری نمونه—`MLP`، `ResBlock`، `InceptionBlock` و `TransformerEncoder`—با موفقیت کامپایل شده و `forward pass` آن‌ها با شکل خروجی صحیح اجرا شده است.

فصل ۱

مقدمه

۱.۱ انگیزه

نوشتن زیرکلاس‌های `nn.Module` در PyTorch به صورت دستی برای معماری‌های پیچیده، تکراری و مستعد خطاست. برای یک شبکه با ده لایه، برنامه‌نویس باید:

- در `__init__` هر لایه را جداگانه با پارامترهای صحیح تعریف کند
 - در `forward` ترتیب اجرا را به صورت دستی مدیریت کند
 - در توپولوژی‌های شاخه‌دار، متغیرهای میانی را نام‌گذاری کند
 - مطمئن شود هیچ تنسور مشترکی پیش از استفاده بازنویسی نمی‌شود
- کد PyTorch زیر را در نظر بگیرید که باید به صورت دستی نوشته شود:

قطعه کد ۱.۱.۱ > نمونه boilerplate دستی برای یک ResBlock

```
1 class ResBlock(nn.Module):
2     def __init__(self):
3         super(ResBlock, self).__init__()
4         self.conv1 = nn.Conv2d(
5             in_channels=64, out_channels=64,
6             kernel_size=3, stride=1, padding=1)
7         self.bn1 = nn.BatchNorm2d(num_features=64)
8         self.relu1 = nn.ReLU()
9         # ... and so on for each layer
```

```
10
11     def forward(self, x):
12         identity = x
13         x = self.conv1(x)
14         x = self.bn1(x)
15         x = self.relu1(x)
16         # ... manage residual connection manually
17         x = x + identity
18         return x
```

با NNGraph DSL، معماری یک‌بار به صورت گراف توصیف می‌شود و کامپایلر این کد را تولید می‌کند.

۲.۱ هدف

هدف NNGraph DSL این است که فایل `nng`. تنها منبع حقیقت معماری باشد. برنامه‌نویس گراف را توصیف می‌کند؛ کامپایلر تمام boilerplate را تولید می‌کند.

۳.۱ دامنه کاربرد

این ابزار برای توصیف معماری‌های ثابت مناسب است—شبکه‌هایی که گراف محاسباتی آن‌ها در زمان تعریف مدل مشخص است. پشتیبانی از کنترل جریان پویا درون `forward` در نسخه‌های آینده برنامه‌ریزی شده است.

فصل ۲

کارهای مرتبط

۱.۲ چارچوب NADER

تعریف ۱.۱.۲ > NADER

NADER (Neural Architecture Design via Representation) چارچوبی است که معماری شبکه‌های عصبی را به صورت گراف محاسباتی جهت‌دار (DAG) مدل می‌کند. هر گره یک عملیات لایه است و هر یال جریان تنسور را نشان می‌دهد.

NNGraph DSL این مدل گراف را به عنوان پایه طراحی زبان به کار می‌گیرد.

۲.۲ ANTLR

تعریف ۱.۲.۲ > ANTLR

ANTLR (ANOther Tool for Language Recognition) ابزاری است که از یک فایل گرامر g4 . به صورت خودکار lexer، parser و listener/visitor تولید می‌کند. نسخه مورد استفاده: ANTLR ۴.۱۳.۲.

۱.۲.۲ الگوی Listener در برابر Visitor

ANTLR^۴ دو الگو برای پیمایش درخت تجزیه ارائه می‌دهد:

• **Listener**: ParseTreeWalker به صورت خودکار درخت را پیمایش می‌کند و متدهای `enter*/exit*`

را فراخوانی می‌کند. هیچ مقدار بازگشتی نیاز نیست.

• **Visitor**: برنامه‌نویس پیمایش را کنترل می‌کند و هر متد یک مقدار برمی‌گرداند.

این کامپایلر از الگوی Listener استفاده می‌کند. متدهای `exit*` در `custom_listener.py` حالت را در `self.nodes` و `self.edges` انباشته می‌کنند—این رویکرد برای ساختن مدل گراف از درخت تجزیه طبیعی‌تر است.

۳.۲ ابزارها و تجسم‌سازهای AST

این کامپایلر از ابزارهای استاندارد درخت نحو انتزاعی (AST) برای بازرسی اختیاری و تجسم استفاده می‌کند. سه فایل برای این منظور به کار می‌روند:

• `ast.py` – کلاس AST و `TreeNode`

• `make_ast_subtree.py` – ترکیب گره‌های درخت تجزیه

• `ast_to_networkx_graph.py` – تجسم اختیاری گراف

۱.۳.۲ چرا AST در اینجا استفاده نمی‌شود

اکثر کامپایلرهای سنتی برنامه‌ها را به صورت درخت نحو انتزاعی (AST) نمایش می‌دهند. زبان‌های مبتنی بر عبارت—که عملگرها در آن‌ها اولویت دارند و محدوده‌ها سلسله‌مراتبی تودرتو می‌شوند—به صورت طبیعی به ساختار درختی نگاشت می‌شوند.

اما `NNGraph DSL` معماری‌های شبکه‌های عصبی را به صورت گراف‌های جهت‌دار غیردوری (DAG) توصیف می‌کند. گره‌ها نشان‌دهنده عملیات لایه‌ها هستند؛ یال‌ها نشان‌دهنده جریان تنسور. ساختار گراف، مدل دامنه است، نه درخت تجزیه.

کامپایلر چندین عملیات مبتنی بر گراف انجام می‌دهد که به لیست مجاورت و روابط صریح یال نیاز دارند:

- **مرتب‌سازی توپولوژیک** (الگوریتم Kahn) برای ترتیب اجرا در `( ) forward`
- **تشخیص دور** از طریق مرتب‌سازی توپولوژیک (اعتبارسنجی DAG)
- **تحلیل دسترس‌پذیری** از طریق BFS از ورودی تا خروجی
- **اعتبارسنجی اتصال باقی‌مانده** (بررسی درجه ورودی گره‌های ادغام)
- **استنتاج شکل** با انتشار شکل تنسورها در امتداد یال‌ها
- **صادرات Graphviz DOT** برای تجسم معماری
- **صادرات ONNX** (ONNX از فرمت گراف استفاده می‌کند)

یک AST سلسله‌مراتب درخت تجزیه (`node_decl`, `graph_block`, `model_block`) را کدگذاری می‌کند، اما گراف محاسباتی را که زبان برای بیان آن طراحی شده است، مبهم می‌سازد. به همین دلیل،

تحلیل‌گر معنایی گره‌ها و یال‌ها را مستقیماً در دیکشنری‌ها و لیست‌ها ذخیره می‌کند—نمایش طبیعی برای الگوریتم‌های گراف.

کد ساخت AST (`exitStart, exitEveryRule`) در `custom_listener.py` برای تجسم اختیاری دیباگ از طریق `show-ast --` باقی می‌ماند. در هنگام کامپایل استفاده نمی‌شود.

فصل ۳

طراحی زبان

۱.۳ ساختار برنامه

هر فایل nng . از سه بلوک تشکیل شده است. بلوک config اختیاری است:

قطعه کد ۱.۱.۳ > ساختار کلی یک فایل nng.

```
1 model <Name> {
2     input  <id> : tensor(<shape>)
3     output <id>
4 }
5
6 graph {
7     node <id> : <LayerType>(<params>)
8     ...
9     edge <src> -> <dst>
10    edge <src> -> <dst> [label="<string>"]
11    ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
17     loss        = "<LossFn>"          // enables training
18     ↪ scaffold
```

```

18     optimizer = "<Optimizer>"    // Adam | SGD | AdamW |
      ↪ RMSprop
19     lr        = <float>          // learning rate
20     epochs    = <int>            // training epochs
21 }

```

تعریف ۲.۱.۳ > بلوک model

بلوک model نام کلاس تولیدشده، شناسه گره ورودی و شکل تانسور ورودی، و شناسه گره خروجی را مشخص می‌کند.

تعریف ۳.۱.۳ > بلوک graph

بلوک graph تمام گره‌های لایه و یال‌های جهت‌دار را تعریف می‌کند. ترتیب اعلان یال‌ها اهمیتی ندارد؛ کامپایلر ترتیب اجرا را از ساختار گراف محاسبه می‌کند.

تعریف ۴.۱.۳ > بلوک config

بلوک config مقادیری را برای بلوک __main__ کد تولیدشده تعیین می‌کند. batch_size پیش‌فرض ۱ و device پیش‌فرض 'cpu' است. اگر کلید loss تنظیم شده باشد، مولد کد یک حلقه آموزش کامل با بهینه‌ساز و تابع خطا تولید می‌کند.

۲.۳ گرامر

تعریف ۱.۲.۳ > آمار گرامر NNGraph.g

- فایل گرامر: NNGraph.g4 (۵۶ خط)
- قوانین تجزیه‌گر: ۱۴ قانون (snake_case)
- توکن‌های واژه‌نگار: ۲۰ توکن (ALL_CAPS)

قوانین کامل گرامر:

قطعه کد ۲.۲.۳ > گرامر NNGraph.g

```

1 grammar NNGraph;
2 start      : model_block graph_block config_block? EOF;

```

```

3 model_block : MODEL ID '{' input_decl output_decl '}';
4 input_decl  : INPUT ID ':' TENSOR shape_expr;
5 output_decl : OUTPUT ID;
6 graph_block : GRAPH '{' (node_decl | edge_decl)* '}';
7 node_decl   : NODE ID ':' layer_expr;
8 layer_expr  : ID '(' param_list? ')';
9 param_list  : param (',' param)*;
10 param       : ID '=' value;
11 edge_decl   : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block: CONFIG '{' config_entry* '}';
13 config_entry: ID '=' value;
14 value       : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15             | STRING | NONE | shape_expr;
16 shape_expr  : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

نکته مهم طراحی: input_decl از shape_expr استفاده می‌کند نه TENSOR. زیرا shape_expr خود پرانتزها را در بر می‌گیرد. این خطای رایج در نسخه‌های اولیه گرامر برطرف شد.

۳.۳ سیستم انواع

مثال	نوع Python	نوع در DSL
128	int	int
1.0	float	float
false / true	bool	bool
"relu"	str	string
(64, 32, 32)	tuple[int]	shape
None	NoneType	None

تمایز بین int و float در زمان کامپایل بررسی می‌شود. مثلاً Dropout (p=1) خطا می‌دهد چون p انتظار float دارد نه int.

۴.۳ انواع لایه

۱.۴.۳ لایه‌های پارامتردار

این لایه‌ها در `__init__` به صورت `self.<id> = nn.X(...)` تولید می‌شوند.

کلیدواژه DSL	کلاس PyTorch	پارامترهای اصلی
Linear	<code>nn.Linear</code>	<code>bias, out_features, in_features</code>
Conv2d	<code>nn.Conv2d</code>	<code>,stride, kernel, out_ch, in_ch, padding</code>
Conv1d	<code>nn.Conv1d</code>	<code>stride, kernel, out_ch, in_ch</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>stride, kernel</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>stride, kernel</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>end_dim, start_dim</code>
Embedding	<code>nn.Embedding</code>	<code>embedding_dim, num_embeddings</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>num_heads, embed_dim</code>
LSTM	<code>nn.LSTM</code>	<code>,hidden_size, input_size, num_layers</code>
GRU	<code>nn.GRU</code>	<code>hidden_size, input_size</code>

۲.۴.۳ فعال‌سازی‌ها

`.ELU(alpha)`, `.LeakyReLU(negative_slope)`, `.Softmax(dim)`, `.GELU`, `.Tanh`, `.Sigmoid`, `.ReLU`

۳.۴.۳ عملیات گراف (بدون مائول)

این انواع در `__init__` خطی تولید نمی‌کنند:

کلیدواژه	پارامترها	کد تولیدشده
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a,b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int, dim: int</code>	<code>split_0, split_1, ... = chunks</code> <code>torch.chunk(x,n,dim=d)</code>

۴.۴.۳ پارامترهای سراسری

پارامتر `quant` روی هر گره بدون توجه به نوع لایه قابل استفاده است. مقادیر مجاز: `"dynamic"`، `"static"`، `"qat"`.

قطعه کد ۱.۴.۳ > نمونه استفاده از کوانتیزاسیون

```
1 node fc1 : Linear(in_features=784, out_features=128,
  ↪ quant="dynamic")
```

۵.۳ نحو یال

تعریف ۱.۵.۳ > یال ساده

```
edge src -> dst
```

یک یال جهت‌دار از گره `src` به گره `dst` تعریف می‌کند.

تعریف ۲.۵.۳ > یال برچسب‌دار

```
edge src -> dst [label="name"]
```

یک یال با برچسب رشته‌ای تعریف می‌کند. برچسب‌ها دو نقش دارند:

- `"shortcut"`: یال اتصال کوتاه به گره `Residual()` را مشخص می‌کند.
- `"residual_*`: اتصال کوتاه به گره‌ای با دو ورودی (الگوی `Transformer`).

۶.۳ مثال کامل: MLP

فایل `nng`. برای یک شبکه سه‌لایه:

قطعه کد ۱.۶.۳ > mlp.nng

```

1  model MLP {
2      input x    : tensor(784)
3      output out
4  }
5
6  graph {
7      node fc1    : Linear(in_features=784, out_features=256)
8      node relu1  : ReLU()
9      node fc2    : Linear(in_features=256, out_features=128)
10     node relu2  : ReLU()
11     node drop   : Dropout(p=0.3)
12     node fc3    : Linear(in_features=128, out_features=10)
13     node out    : Softmax(dim=1)
14
15     edge x      -> fc1
16     edge fc1    -> relu1
17     edge relu1  -> fc2
18     edge fc2    -> relu2
19     edge relu2  -> drop
20     edge drop   -> fc3
21     edge fc3    -> out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27 }

```

کد PyTorch تولیدشده:

قطعه کد ۲.۶.۳ > کد PyTorch تولیدشده برای MLP

```

1  import torch
2  import torch.nn as nn

```

```

3
4
5 class MLP(nn.Module):
6     def __init__(self):
7         super(MLP, self).__init__()
8         self.fc1 = nn.Linear(in_features=784,
9                               ↪ out_features=256)
10        self.relu1 = nn.ReLU()
11        self.fc2 = nn.Linear(in_features=256,
12                              ↪ out_features=128)
13        self.relu2 = nn.ReLU()
14        self.drop = nn.Dropout(p=3.0)
15        self.fc3 = nn.Linear(in_features=128,
16                              ↪ out_features=10)
17        self.out = nn.Softmax(dim=1)
18
19    def forward(self, x):
20        x = self.fc1(x)
21        x = self.relu1(x)
22        x = self.fc2(x)
23        x = self.relu2(x)
24        x = self.drop(x)
25        x = self.fc3(x)
26        x = self.out(x)
27        return x
28
29 if __name__ == '__main__':
30     device = torch.device('cuda')
31     model = MLP().to(device)
32     x = torch.randn(64, 784).to(device)
33     print(model(x).shape)

```


فصل ۴

پیاده‌سازی

۱.۴ معماری کامپایلر

تعریف ۱.۱.۴ > خط لوله کامپایل

کامپایلر شش مرحله دارد:

۱. تجزیه: `FileStream` → `NNGraphLexer` → `NNGraphParser.program()`

درخت تجزیه

۲. تحلیل معنایی: `NNGraphCustomListener + ParseTreeWalker` → مدل گراف

۳. مرتب‌سازی توپولوژیک: محاسبه ترتیب اجرا با الگوریتم Kahn

۴. استنتاج شکل: انتشار شکل تنسور در گراف و تشخیص عدم تطابق ابعاد (اختیاری):

(`--show-shapes`)

۵. تولید کد: `CodeGenerator.generate()` → رشته Python

۶. خروجی: نوشتن به فایل `.py`، خروجی `Graphviz DOT`، یا صادرکردن `ONNX`

۱.۱.۴ مسئولیت فایل‌ها

فایل	نقش
NNGraph.g4	منبع حقیقت گرامر (۵۶ خط)
gen/NNGraphLexer.py	تولیدشده توسط ۴ANTLR
gen/NNGraphParser.py	تولیدشده توسط ۴ANTLR
gen/NNGraphListener.py	listener پایه – نباید ویرایش شود
custom_listener.py	تحلیل معنایی
code_generator.py	تولید کد PyTorch
shape_inference.py	استنتاج شکل تنسور در کامپایل
graph_viz.py	تولید نمودار DOT Graphviz
onnx_export.py	صادرکردن مدل به فرمت ONNX
main.py	رابط CLI و compile_nng()

۲.۴ تحلیل‌گر معنایی

تعریف ۱.۲.۴ > NNGraphCustomListener

کلاس NNGraphCustomListener(NNGraphListener) در custom_listener.py تمام منطق تحلیل معنایی را پیاده می‌کند. حالت داخلی:

```
dict[id, {type, params, line}]:self.nodes •
list[{src, dst, label, line}]:self.edges •
dict[key, value]:self.config •
```

۱.۲.۴ انتشار مقدار در درخت

متدهای exitShape_expr و exitValue مقادیر را روی خود context ذخیره می‌کنند:

قطعه کد ۲.۲.۴ > انتشار مقدار ctx.pvalue در exitShape_expr

```
1 def exitShape_expr(self, ctx):
2     ints = [int(ctx.INT_LITERAL(i).getText())
```

```

3         for i in range(ctx.INT_LITERAL().__len__())
4             ctx.pvalue = tuple(ints)
5             ctx.ptype = tuple

```

این رویکرد تضمین می‌کند که هر context پس از خروج از آن، مقادیر فرزندانش را در دسترس دارد.

۲.۲.۴ قوانین اعتبارسنجی

هشت قانون اعتبارسنجی در دو فاز اجرا می‌شوند:

فاز اول (در زمان پیمایش درخت):

۱. شناسه تکراری گره: در `exitNode_decl`

۲. ارجاع نامعتبر در یال: در `exitEdge_decl`

۳. نوع لایه ناشناس: در `exitNode_decl`

۴. عدم تطابق نوع پارامتر: در `exitNode_decl`

فاز دوم (در `exitGraph_block`، پس از تکمیل گراف):

۵. گره خروجی اعلان نشده: در `_validate_output_declared`

۶. گره یتیم: در `_validate_orphans`

۷. آریتی `Residual != ۲`: در `_validate_residual_arity`

۸. تشخیص دور با `Kahn`: در `_validate_no_cycles`

۹. قابلیت دسترسی `BFS`: در `_validate_reachability`

۳.۲.۴ قالب پیام خطا

قطعه کد ۳.۲.۴ > قالب خطا

```

1 [line L:C] SemanticError: <message>
2 [line L] ShapeError at '<node>': <message>

```

L شماره سطر (از ۱) و C ستون توکن (از ۰) است. تمام شرایط خطا کامپایل را متوقف می‌کنند و با کد خروج ۱ پایان می‌یابند.

۴.۲.۴ FakeCtx برای خطاهای پس از پیمایش

خطاهایی که در `exitGraph_block` رخ می‌دهند context زنده ندارند. یک کلاس داخلی `_FakeCtx` شماره سطر را از رکورد اعلان گره حمل می‌کند:

قطعه کد ۴.۲.۴ > کلاس `_FakeCtx` برای حفظ شماره خط

```
1 class _FakeCtx:
2     class start:
3         line    = info["line"]
4         column  = 0
```

۵.۲.۴ جریان اجرای شنونده

متد `ParseTreeWalker.walk(listener, parse_tree)` یک پیمایش عمق‌اول (DFS) روی درخت تجزیه انجام می‌دهد. برای هر گره، متد `enterX(ctx)` را قبل از بازدید فرزندان و متد `exitX(ctx)` را پس از بازدید تمامی فرزندان فراخوانی می‌کند.

تعریف ۵.۲.۴ > فراخوانی واکر در `main.py`

```
1 NNGraphCustomListener() = listener
2 ruleNames.parser = rule_names.listener
3 ParseTreeWalker() = walker
4 parse_tree), walk(listener.walker
```

شنونده سفارشی تنها متدهای `*exit` را بازنویسی می‌کند. تمامی متدهای `*enter` به صورت بدون عملیات (no-op) از کلاس پایه `NNGraphListener` باقی می‌مانند.

انتشار ویژگی‌ها (پایین به بالا)

مقادیر از طریق ویژگی‌های کانتکست (`context attributes`) به سمت بالا در درخت جریان می‌یابند:

قطعه کد ۶.۲.۴ > متد `exitValue` مقدار پارس‌شده را در `ctx.pvalue` ذخیره می‌کند

```
1 def exitValue(self, ctx):
2     if ctx.FLOAT_LITERAL():
3         ctx.pvalue = float(ctx.FLOAT_LITERAL().getText())
4         ctx.ptype  = float
5     elif ctx.INT_LITERAL():
```

```

6         ctx.pvalue = int(ctx.INT_LITERAL().getText())
7         ctx.ptype  = int
8     elif ctx.BOOL_LITERAL():
9         ctx.pvalue = ctx.BOOL_LITERAL().getText() == 'true'
10        ctx.ptype  = bool
11    # ... other cases

```

قوانین والد ویژگی‌های فرزندان را می‌خوانند:

قطعه کد ۷.۲.۴ > متد exitParam مقدار ctx.value().pvalue را از فرزند می‌خواند

```

1 def exitParam(self, ctx):
2     ctx.param_name = ctx.ID().getText()
3     ctx.param_value = ctx.value().pvalue
4     ctx.param_type  = ctx.value().ptype

```

استخراج معنایی و شرح تفصیلی رفتار متدها

شنونده سفارشی بر مجموعه‌ای از متدهای *exit برای پردازش درخت تجزیه به صورت پایین به بالا تکیه دارد. در ادامه شرح تفصیلی نحوه عملکرد تمامی توابع شنونده آورده شده است:

- **exitShape_expr(ctx)**: لیستی از ابعاد را از توکن‌های INT_LITERAL پارس می‌کند. یک tuple پایتونی ساخته و آن را به ctx.pvalue نسبت می‌دهد و ctx.ptype را برابر tuple قرار می‌دهد.
- **exitValue(ctx)**: مقادیر لیترال (INT_LITERAL، FLOAT_LITERAL، BOOL_LITERAL، NONE، STRING یا shape_expr تودرتو) را ارزیابی می‌کند. متن منطبق را به نمایش پایتونی معادل آن تبدیل کرده، نتیجه را به ctx.pvalue و نوع را به ctx.ptype اختصاص می‌دهد.
- **exitModel_block(ctx)**: شناسه مدل شبکه عصبی را از طریق ctx.ID().getText() استخراج کرده و در self.model_name ذخیره می‌کند.
- **exitInput_decl(ctx)**: شناسه متغیر ورودی را در self.input_id ذخیره کرده و شکل پارس‌شده را به self.input_shape منتقل می‌کند. علاوه بر این، self.nodes را با یک شبه‌گره از نوع "__input__" در شماره خط فعلی پر می‌کند تا امکان اعتبارسنجی یال‌های خروجی از ورودی فراهم شود.
- **exitOutput_decl(ctx)**: شناسه متغیر خروجی را در self.output_id ذخیره می‌کند تا گره پایانی شبکه مشخص باشد.

- **exitParam(ctx)**: پیکربندی‌های پارامتر کلید-مقدار را پردازش می‌کند. شناسه پارامتر را به `ctx.param_name` و مقادیر و انواع ارزیابی‌شده را به ترتیب به `ctx.param_value` و `ctx.param_type` نسبت می‌دهد.
- **exitLayer_expr(ctx)**: پیکربندی‌های سازنده لایه را پردازش می‌کند. نام کلاس لایه را از روی شناسه تشخیص داده و دیکشنری نگاشت نام پارامترها به مقادیر و انواع پارس‌شده‌شان را ساخته و به `ctx.params` نسبت می‌دهد.
- **exitNode_decl(ctx)**: گره‌ها را ثبت و محدودیت‌های ساختاری را اعتبارسنجی می‌کند. اطمینان حاصل می‌کند که شناسه گره قبلاً اعلان نشده باشد. نوع لایه را با `LAYER_SCHEMA` مطابقت می‌دهد، تطابق هر پارامتر اعلان‌شده با مشخصات طرح‌واره (نوع و فرمت) را بررسی می‌کند، معتبر بودن حالت کوانتیزاسیون اختیاری (`quant`) را بررسی کرده و در نهایت گره را در `self.nodes` ثبت می‌کند.
- **exitEdge_decl(ctx)**: اتصالات بین گره‌ها را پردازش می‌کند. شناسه‌های مبدأ و مقصد را استخراج کرده، وجود هر دو گره در `self.nodes` را اعتبارسنجی می‌کند، برچسب‌های اختیاری (مانند "shortcut" یا "residual_*") را بررسی کرده و یال را در `self.edges` ثبت می‌کند.
- **exitConfig_entry(ctx)**: تنظیمات سراسری کامپایلر (مانند سخت‌افزار هدف `device` یا اندازه بچ `batch_size`) را استخراج کرده و دیکشنری `self.config` را پر می‌کند.
- **exitGraph_block(ctx)**: پس از پیمایش کامل بلوک گراف فراخوانی می‌شود. پنج فرآیند اعتبارسنجی متوالی را اجرا می‌کند: بررسی اعلان گره خروجی، شناسایی گره‌های یتیم، بررسی درجه ورودی گره `Residual` (باید دقیقاً ۲ باشد)، اعتبارسنجی بدون دور بودن گراف، و تأیید قابلیت دسترسی مسیر از ورودی به خروجی.
- **exitEveryRule(ctx)**: خروج از تمامی قوانین را ردیابی می‌کند. اگر نام قوانین لود شده باشد، `make_ast_subtree` را فراخوانی می‌کند تا درخت‌های نحو انتزاعی را به صورت پایین به بالا بسازد و ادغام کند.
- **exitStart(ctx)**: ساختار درخت نحو انتزاعی (AST) را نهایی کرده و درخت کامل برنامه را به بخش جمع‌آوری AST شنونده متصل می‌کند.

الگوریتم‌های اعتبارسنجی گراف

متد `exitGraph_block` پس از ساخت کامل گراف، پنج مرحله اعتبارسنجی را اجرا می‌کند:

۱. تشخیص گره یتیم (`_validate_orphans`):

قطعه کد ۸.۲.۴ الگوریتم تشخیص گره یتیم

```

1 referenced = set()
2 for e in self.edges:
3     referenced.add(e["src"])
4     referenced.add(e["dst"])
5 for node_id, info in self.nodes.items():
6     if info["type"] == "__input__":
7         continue
8     if node_id not in referenced:
9         error(f"Node '{node_id}' is declared but never
            → referenced.")

```

شناسه تمامی گره‌های به کار رفته در یال‌ها را جمع‌آوری می‌کند. هر گره اعلان‌شده‌ای (به جز ورودی) که در این مجموعه نباشد، یتیم است—یعنی اعلان شده اما هیچ‌گاه وصل نشده است.

۲. تشخیص دور (`_validate_no_cycles`):

از الگوریتم مرتب‌سازی توپولوژیک Kahn استفاده می‌کند. یک گراف جهت‌دار بدون دور (DAG) می‌تواند کاملاً مرتب شود؛ اگر پس از پردازش، گرهی با $\text{in_degree} > 0$ باقی بماند، یک دور وجود دارد.

قطعه کد ۹.۲.۴ الگوریتم Kahn برای تشخیص دور

```

1 in_degree = dict.fromkeys(self.nodes, 0)
2 adj       = {n: [] for n in self.nodes}
3 for e in self.edges:
4     adj[e["src"]].append(e["dst"])
5     in_degree[e["dst"]] += 1
6
7 queue = deque(n for n, d in in_degree.items() if d == 0)
8 visited = 0
9 while queue:
10     node = queue.popleft()
11     visited += 1
12     for nxt in adj[node]:
13         in_degree[nxt] -= 1
14         if in_degree[nxt] == 0:
15             queue.append(nxt)

```

```

16
17 if visited != len(self.nodes):
18     remaining = [n for n, d in in_degree.items() if d > 0]
19     error(f"Cycle detected involving nodes: {remaining}.")

```

۳. قابلیت دسترسی (`_validate_reachability`):

یک جستجوی عرض‌اول (BFS) از گره ورودی انجام می‌دهد. تک‌تک گره‌های اعلان‌شده و گره خروجی باید قابل دسترسی باشند.

قطعه کد ۱۰.۲.۴ > بررسی قابلیت دسترسی با BFS

```

1 adj = {n: [] for n in self.nodes}
2 for e in self.edges:
3     adj[e["src"]].append(e["dst"])
4
5 visited = set()
6 queue = deque([self.input_id])
7 while queue:
8     node = queue.popleft()
9     if node in visited:
10        continue
11    visited.add(node)
12    for nxt in adj[node]:
13        queue.append(nxt)
14
15 for node_id, info in self.nodes.items():
16     if node_id not in visited and info["type"] !=
17         ↳ "__input__":
18         error(f"Node '{node_id}' is not reachable from
19             ↳ input.")
20
21 if self.output_id not in visited:
22     error(f"Output node '{self.output_id}' is not
23         ↳ reachable.")

```

بخش‌های جدا افتاده و مسیرهای قطع‌شده بین ورودی و خروجی را شناسایی می‌کند.

تشخیص دور و بررسی قابلیت دسترسی، ارزیابی‌های مستقلی هستند. یک گراف می‌تواند بدون دور باشد اما قابلیت دسترسی آن خراب باشد (اجزای ناپیوسته)، یا متصل باشد اما دارای دور باشد. هر دو ویژگی برای یک DAG معتبر شبکه عصبی الزامی هستند.

۳.۴ مولد کد

تعریف ۱.۳.۴ > CodeGenerator

کلاس CodeGenerator در code_generator.py چهار گام داخلی دارد:

۱. `_compute_graph()`: ساخت لیست‌های مجاورت و مرتب‌سازی توپولوژیک
۲. `_assign_vars()`: تخصیص نام متغیر Python به هر گره
۳. `_emit_forward() + _emit_init()`: تولید خطوط کد
۴. `_assemble()`: ترکیب در سورس Python نهایی

۴.۴ الگوریتم تخصیص متغیر

تعریف ۱.۴.۴ > اصل اساسی تخصیص متغیر

`var_at[n] = 'x'` فقط وقتی که `out_degree[predecessor] == 1` وقتی یک گره به چند مقصد فرستاده می‌شود (`out_degree > 1`)، تنسور آن هنوز توسط شاخه‌های دیگر نیاز است. بازنویسی `x` در این نقطه آن را خراب می‌کند. به جای آن، گره بعدی نام شناسه خودش را به عنوان متغیر دریافت می‌کند.

چهار قانون کامل در `_assign_vars`:

قطعه کد ۲.۴.۴ > قوانین تخصیص متغیر در `_assign_vars`

```

1 Rule 1 - Input node:
2     var_at[input_id] = 'x'
3
4 Rule 2 - Merge node (in_degree > 1):
5     var_at[n] = 'x'         if out_degree[n] <= 1
6     var_at[n] = node_id    if out_degree[n] > 1
7

```

```

8 Rule 3 - Branch start (predecessor.out_degree > 1):
9     var_at[n] = node_id
10
11 Rule 4 - Linear continuation (predecessor.out_degree == 1):
12     var_at[n] = var_at[predecessor]
```

۱.۴.۴ ردیابی MLP (زنجیره خطی)

مثال ۳.۴.۴ ردیابی متغیر برای MLP

توپولوژی:

$x \rightarrow fc1 \rightarrow relu1 \rightarrow fc2 \rightarrow relu2 \rightarrow drop \rightarrow fc3 \rightarrow out$

هر predecessor دارای $out_degree == 1$ است $\Rightarrow$ قانون ۴ همیشه اعمال می‌شود $\Rightarrow$ همه گره‌ها $var_at = 'x'$ دارند. کد forward از x در همه جا استفاده می‌کند.

۲.۴.۴ ردیابی ResBlock (فناوت سپس ادغام)

مثال ۴.۴.۴ ردیابی متغیر برای ResBlock

x به $conv1$ و $skip$ (اتصال کوتاه) فرستاده می‌شود. پس $out_degree[x] = 2$.
 $conv1$: قانون ۳ $\Rightarrow var_at = 'conv1'$
 $bn1, relu1, conv2, bn2$: قانون ۴ $\Rightarrow$ همه $'conv1'$
 $var_at = 'x' \Rightarrow out_degree=1, 2$: قانون ۲ (Residual, skip): $in_degree=2$
 کد تولیدشده:

قطعه کد ۵.۴.۴ pass forward تولیدشده برای ResBlock

```

1 conv1 = self.conv1(x)
2 conv1 = self.bn1(conv1)
3 conv1 = self.relu1(conv1)
4 conv1 = self.conv2(conv1)
5 conv1 = self.bn2(conv1)
6 x     = conv1 + x      # Residual node
7 x     = self.flatten(x)
```

۳.۴.۴ ردیابی InceptionBlock (دو شاخه موازی)

مثال ۶.۴.۴ ردیابی متغیر برای InceptionBlock

$\text{out_degree}[x] = 2$ (به convA1 و convB1).
 • convA1: قانون ۳ $\Rightarrow$ 'convA1'
 • convB1: قانون ۳ $\Rightarrow$ 'convB1'
 • convA2: قانون ۴ $\Rightarrow$ ارث از 'convA1'
 • convB2: قانون ۴ $\Rightarrow$ ارث از 'convB1'
 • (Concat, cat1): $\text{in_degree}=2$: قانون ۲ $\Rightarrow$ 'x'
 کد تولیدشده:

قطعه کد ۷.۴.۴ pass forward تولیدشده برای InceptionBlock

```

1 convA1 = self.convA1(x)
2 convA1 = self.convA2(convA1)
3 convB1 = self.convB1(x)
4 convB1 = self.convB2(convB1)
5 x      = torch.cat([convA1, convB1], dim=1)  # Concat
   ↪ node
6 x      = self.bn(x)
7 x      = self.out(x)
  
```

۴.۴.۴ ردیابی TransformerEncoder (دو residual ضمنی)

مثال ۸.۴.۴ ردیابی متغیر برای TransformerEncoder

$\text{out_degree}[\text{norm1}] = 2$ و $\text{out_degree}[x] = 2$
 • attn: قانون ۳ $\Rightarrow$ 'attn'
 • drop1: قانون ۴ $\Rightarrow$ 'attn'
 • norm1: $(\text{in_deg}=2, \text{out_deg}=2)$: قانون ۲ $\Rightarrow$ 'norm1'
 • ff1: قانون ۳ $\Rightarrow$ 'ff1'
 • gelu1, drop2, ff2: قانون ۴ $\Rightarrow$ 'ff1'
 • norm2: $(\text{in_deg}=2, \text{out_deg}=1)$: قانون ۲ $\Rightarrow$ 'x'
 کد تولیدشده:

قطعه کد ۹.۴.۴ > pass forward تولیدشده برای TransformerEncoder

```

1  attn, _ = self.attn(x, x, x)      # MultiHeadAttn
   ↳ triple-pass
2  attn      = self.drop1(attn)
3  norm1     = self.norm1(x + attn)  # implicit residual_1
4  ff1       = self.ff1(norm1)
5  ff1       = self.gelu1(ff1)
6  ff1       = self.drop2(ff1)
7  ff1       = self.ff2(ff1)
8  x         = self.norm2(norm1 + ff1) # implicit
   ↳ residual_2
9  x         = self.out(x)
10 x         = self.flatten(x)
11 return x

```

۵.۴.۴ موارد خاص تولید کد

الگو	شرط	کد تولیدشده
MultiHeadAttn	نوع لایه	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	نوع لایه	<code>out, _ = self.lstm(x)</code>
Residual	بدون ماژول	<code>out = main + shortcut</code>
Add	بدون ماژول	<code>out = a + b</code>
Concat	بدون ماژول	<code>out = torch.cat([a,b], dim=d)</code>
Split	بدون ماژول	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>
Residual ضمنی	<code>label residual_*</code>	<code>out = self.norm(skip + main)</code>

نگاشت نام پارامتر: DSL از نام‌های کوتاه‌تر استفاده می‌کند. مولد کد آن‌ها را به نام‌های PyTorch نگاشت می‌کند: `in_ch → in_channels, out_ch → out_channels, kernel → kernel_size`

۶.۴.۴ ساختار کد خروجی

قطعه کد ۱۰.۴.۴ الگوی کد خروجی تولیدشده

```

1 import torch
2 import torch.nn as nn
3
4
5 class <ModelName>(nn.Module):
6     def __init__(self):
7         super(<ModelName>, self).__init__()
8         # one line per parameterized layer node
9
10    def forward(self, x):
11        # one line per node in topological order
12        return <output_var>
13
14
15 if __name__ == '__main__':
16     device = torch.device('<device>')
17     model = <ModelName>().to(device)
18     x = torch.randn(<batch_size>,
19                    ↪ <input_shape>).to(device)
20     print(model(x).shape)

```

۵.۴ استنتاج شکل

تعریف ۱.۵.۴ Inference Shape

ماژول `shape_inference.py` شکل تنسور را از گره ورودی در امتداد مرتب‌سازی توپولوژیک انتشار می‌دهد. برای هر نوع لایه، قوانین خاصی اعمال می‌شود: `Linear` بعد آخر را تغییر می‌دهد، `Conv2d` ابعاد فضایی را محاسبه می‌کند، `Flatten` ابعاد را ادغام می‌کند، و غیره.

در صورت عدم تطابق ابعاد، خطای `ShapeError` با شماره سطر اعلان گره گزارش می‌شود و کامپایل متوقف می‌شود. پرچم `--show-shapes` شکل خروجی هر گره را نمایش می‌دهد.

۶.۴ تجسم Graphviz

ماژول `graph_viz.py` خروجی Graphviz DOT تولید می‌کند. گره‌ها بر اساس نوع لایه رنگ‌بندی می‌شوند (سبز برای ورودی، آبی برای Linear، نارنجی برای Conv، بنفش برای Normalization). گره خروجی با `peripheries=2` مشخص می‌شود و یال‌های برجسته‌دار با خط‌چین نمایش داده می‌شوند. اگر استنتاج شکل فعال باشد، شکل‌ها روی گره‌ها نوشته می‌شوند.

قطعه کد ۱.۶.۴ > تولید و رندر نمودار

```
1 python main.py -i input/inception.nng --graph-viz
   ↪ output/inception.dot
2 dot -Tpng output/inception.dot -o output/inception.png
```

۷.۴ داربست آموزش

اگر کلید `loss` در بلوک `config` تنظیم شده باشد، مولد کد یک حلقه آموزش کامل شامل تابع خطا، بهینه‌ساز، و حلقه اپوک تولید می‌کند. توابع خطا پشتیبانی‌شده: `MSELoss`، `CrossEntropyLoss`، `SmoothL1Loss`، `NLLoss`، `L1Loss`، `BCEWithLogitsLoss`، `BCELoss`، `RMSprop`، `AdamW`، `SGD`، `Adam`.

۸.۴ حاشیه‌نویسی کوانتیزاسیون

پارامتر سراسری `quant` روی هر گره قابل استفاده است. سه حالت پشتیبانی می‌شود: `"dynamic"`، `"static"` و `"qat"`. گره‌های QAT در `torch.quantization.QuantWrapper` بسته‌بندی می‌شوند. وقتی حداقل یک گره حاشیه‌نویسی کوانتیزاسیون داشته باشد، `QuantStub` و `DeQuantStub` در `__init__` و `forward` اضافه می‌شوند.

۹.۴ صادرکردن ONNX

ماژول `onnx_export.py` کد تولیدشده را نمونه‌سازی کرده و با `torch.onnx.export` یک فایل `onnx` با محور دسته‌ای پویا و `opset ۱۷` تولید می‌کند.

قطعه کد ۱.۹.۴ > صادرکردن مدل به ONNX

```
1 python main.py -i input/mlp.nng --onnx output/mlp.onnx
```

فصل ۵

ارزیابی

۱.۵ نتایج pass forward

مثال ۱.۱.۵ > MLP

- ورودی: `tensor(784)` – خروجی: `(batch, 10)`
- آزمایش: Softmax ردیف‌ها به ۱ جمع می‌شوند ($\text{atol}=1e-5$)

مثال ۲.۱.۵ > ResBlock

- ورودی: `tensor(64, 32, 32)` – خروجی: `(batch, 65536)`
- آزمایش: شکل صحیح؛ در حالت `eval` خروجی قطعی است

مثال ۳.۱.۵ > InceptionBlock

- ورودی: `tensor(32, 56, 56)` – خروجی: `(batch, 48, 56, 56)`
- شاخه A: ۱۶ کانال؛ شاخه B: ۳۲ کانال؛ ادغام با `Concat(dim=1)` → ۴۸ کانال

مثال ۴.۱.۵ > TransformerEncoder

- ورودی: `tensor(512, 128)` – خروجی: `(1024, 128)`
- دو زیرلایه `residual` ضمنی بدون گره `Residual()`
- `MultiHeadAttn` با `embed_dim=128, num_heads=8,` و `batch_first=True`

۲.۵ اثبات درستی متغیرها

تعریف ۱.۲.۵ اثبات عدم بازنویسی تانسور

برای هر نمونه شاخه‌دار، `data_ptr()` تانسور ورودی در طول محاسبات forward ردیابی شد:

- **ResBlock**: `x.data_ptr()` در نقطه `x = conv1 + x` تغییر نکرده
- **Inception**: `x.data_ptr()` هنگام `convA1(x)` و `convB1(x)` یکسان است
- **Transformer**: `x.data_ptr()` هنگام `norm1(x + attn)`
- `norm1.data_ptr()` هنگام `norm2(norm1 + ff1)` تغییر نکرده

این نتیجه مستقیماً از اصل `var_at[n]='x'` فقط وقتی `out_degree=1` پیروی می‌کند.

۳.۵ آمار کدبیس

فایل/دایرکتوری خطوط

۵۶ NNGraph.g4
 ۳۰۳ custom_listener.py
 ۳۲۷ code_generator.py
 ۱۹۳ shape_inference.py
 ۹۰ graph_viz.py
 ۴۷ onnx_export.py
 ۱۲۱ main.py
 ۵۷۵ tests/ (۴ فایل)
 ۱۰۰≈ required_code_collection/

جمع ۱۸۱۲≈

- زبان: Python ۳.۱۴.۳
- ANTLR: ۲.۱۳.۴
- PyTorch: ۰.۱۰.۲

فصل ۶

تصمیمات طراحی

۱.۶ Visitor در برابر Listener

الگوی Listener به صورت طبیعی با انباشت حالت سازگار است. متدهای `exit*` می‌توانند مستقیماً روی `self.nodes` و `self.edges` عمل کنند بدون اینکه مقادیر را از طریق درخت برگردانند.

۲.۶ `load_from_listener()` در برابر `generate(traversal)`

برخی کامپایلرها از `CodeGenerator.generate(traversal)` استفاده می‌کنند که یک لیست توکن پس‌ترتیب دریافت می‌کند—مناسب برای تولید بایت‌کد پشت‌های. برای کامپایلر گراف، ورودی طبیعی ساختار گراف (گره‌ها + یال‌ها) است نه جریان خطی توکن. پس `load_from_listener(listener)` مستقیم‌تر است.

۳.۶ نام‌گذاری متغیر با `node_id`

مثال‌های مشخصات زبان از نام‌های تک‌حرفی یا توصیفی استفاده می‌کنند. این کامپایلر از شناسه واقعی گره استفاده می‌کند زیرا:

- قطعی است و برای گراف‌های بزرگ‌تر بدون شمارنده کار می‌کند
- خوانا است—متغیر `convA1` واضح‌تر از `a` است
- خروجی ریاضی یکسان است

فصل ۷

نتیجه‌گیری و کارهای آینده

۱.۷ نتیجه‌گیری

NNGraph DSL یک زنجیره کامپایلر شش مرحله‌ای از `nng` به `nn.Module` قابل اجرا ارائه می‌دهد. کامپایلر گرامر ANTLR۴ با ۱۴ قانون تجزیه‌گر، ۲۴ نوع لایه، ۸ قانون اعتبارسنجی، و الگوریتم تخصیص متغیر که شاخه‌بندی پیچیده را بدون بازنویسی تنسور مدیریت می‌کند را ارائه می‌دهد. کامپایلر در چهار معماری نمونه اعتبارسنجی شده است.

۲.۷ قابلیت‌های پیاده‌سازی شده

قابلیت	توضیح
استنتاج شکل	انتشار شکل تنسور و تشخیص عدم تطابق در کامپایلر (<code>--show-shapes</code>)
تجسم Graphviz	خروجی نمودار گراف مدل به فرمت DOT (<code>--graph-viz</code>)
داربست آموزش	تولید حلقه آموزش با بهینه‌ساز و تابع خطا از بلوک <code>config</code>
کوانتیزاسیون	متادیتا <code>quant</code> برای تبدیل مدل به نسخه‌های کوانتیزه
صادرکردن ONNX	صادرکردن مدل به فرمت ONNX (<code>--onnx</code>)

۳.۷ کارهای آینده

قابلیت	توضیح
زیرگراف‌های نام‌دار	بلوک‌های ماکرو قابل استفاده مجدد
کنترل جریان پویا	شاخه‌های <code>else/if</code> درون <code>forward</code>

منابع

۱. Parr, T. (۲۰۱۳.) The Definitive ANTLR ۴ Reference. Pragmatic Bookshelf.
۲. Paszke, A., et al. (۲۰۱۹.) PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS ۲۰۱۹.
۳. NADER Framework: Neural Architecture Design via Representation .
۴. Zhang, A. M. amznotes LaTeX template. <https://github.com/alexmingzhang/latex-notes-template>